TRAFFIC-AWARE ROUTE PLANNING USING A* SEARCH

Artificial Intelligence Mini Project Report

Submitted by: Aathan A

Register Number: RA2612049015079

Course / Subject: Fundamentals of Artificial Intelligence

Department: Computer Science and Technology

Institution: SRM Institute of Science and Technology

Submission Date: 27/10/2026

Abstract

This project develops a traffic-aware route-planning system that finds a minimum-travel-time path between two junctions of a road network learned from real-format vehicle data. The 3,565 GPS readings of the Sustainable Vehicle Routing dataset (speed, traffic density, road condition, weather, fuel consumption and CO_2 emission) are mapped onto a 12 × 12 grid of junctions (144 junctions and 506 roads). The cost of each road is the estimated travel time, computed from the average speed observed near its endpoints and a delay factor derived from traffic, road and weather conditions. A* search with an admissible straight-line-time heuristic is compared with Dijkstra's algorithm, breadth-first search (BFS) and greedy best-first search on 300 random trips in each of four scenarios (all traffic, low, medium and high traffic). A* returned the optimal route in 100% of trips while expanding 57% fewer junctions than Dijkstra on average; BFS, which ignores traffic, produced trips that were 13.7% to 15.8% slower and emitted 11.2% to 12.7% more CO_2; greedy search was the fastest but found the optimal route in only 27–33% of trips. The complete experiment is implemented in a single Python program that also produces five charts and a results table.

1. Problem Definition

In a road network, the geographically shortest route is not necessarily the fastest or the most sustainable route. Congestion, wet or icy roads and poor weather increase the time needed to traverse a road, and road closures (for example after an accident) make some links unavailable. The problem is to find the route with the minimum estimated travel time from a start junction to a destination junction while accounting for traffic, road and weather conditions and blocked roads, and to report the fuel use and CO_2 emission of the chosen route.

Input: GPS readings with speed, traffic density, road condition, weather, fuel consumption and CO_2 emission; the junction grid derived from them; a start junction, a destination junction; a traffic scenario; and an optional set of blocked roads. Output: the route (sequence of junctions), total travel time in minutes, distance in km, CO_2 in grams, fuel in litres, number of junctions expanded and search runtime in milliseconds.

2. AI Problem Formulation

- **State:** the junction at which the vehicle is currently located.
- **Initial state:** the start junction chosen for the trip.
- **Goal state:** the destination junction.
- **Actions:** drive to any adjacent junction (horizontal, vertical or diagonal neighbour) along an available road.
- **Transition model:** taking a road moves the vehicle from one endpoint of the road to the other.
- **Path cost:** the estimated travel time of each road, in minutes, accumulated along the route.
- **Constraints:** blocked roads are removed from the set of available actions; every delay factor is at least 1.

The road network is a weighted graph G = (V, E). Each grid cell is a junction; the 12 × 12 grid gives |V| = 144 and 8-neighbour connectivity gives |E| = 506. For a road e joining junctions u and v, let d be its length in km (distance between the cell centres), $\bar{v}$ the mean of the average speeds of the two cells (km/h) and $\bar{p}$ the mean of their average delay penalties. The edge cost is:

$$\mathbf{cost(e) = (d / \bar{v}) \times 60 \times (1 + \bar{p}) \quad minutes}$$

The fuel and CO_2 of a road are d multiplied by the mean fuel consumption (L/km) and mean CO_2 emission (g/km) of the two cells. The delay penalty of each GPS reading is the sum of a traffic, a road-condition and a

weather component (Table 3). Because every component is zero or positive, $1 + \bar{p} \geq 1$ and travel time can only be longer than the free-flow time.

3. Selection and Explanation of AI Technique

3.1 A* Search

A* is an informed graph-search algorithm. It expands the frontier node with the smallest value of f(n) = g(n) + h(n), where g(n) is the best known travel time from the start to n and h(n) estimates the remaining travel time from n to the goal. The heuristic directs the search toward the destination, so fewer junctions are examined than in an uninformed search.

$$\mathbf{f(n) = g(n) + h(n)}$$

$$\mathbf{h(n) = (\ straight\text{-}line\ distance(n, goal)\ /\ v_max\ ) \times 60 \quad minutes}$$

Here v_max is the highest average road speed in the network (49.0 km/h in the all-traffic network). This heuristic is admissible: the straight-line distance cannot exceed the distance travelled along roads, no road is driven faster than v_max, and every delay factor is at least 1, so the true remaining travel time is never smaller than h(n). It is also consistent, because the straight-line time obeys the triangle inequality and each real road costs at least its straight-line time at v_max. Therefore A* returns an optimal (minimum-time) route.

3.2 Baseline Algorithms

- **Dijkstra's algorithm:** A* with h(n) = 0. It uses the same traffic-aware costs and is also optimal, so it shows how much search effort the heuristic saves.
- **Breadth-first search (BFS):** finds the route with the fewest junctions and ignores travel time and traffic. It shows the cost of planning without traffic information.
- **Greedy best-first search:** orders the frontier only by h(n). It is very fast but not optimal, because it ignores the cost already incurred.

4. Python Implementation

The complete project is a single Python program, traffic_route_optimization.py (also supplied as a Jupyter notebook with identical logic). It depends on pandas, numpy and matplotlib only, and reads sustainable_vehicle_routing.csv from the same folder. Its main components are:

- **load_data / add_cells:** reads the CSV, removes duplicates and missing values, corrects negative fuel values, computes the delay penalty of every reading and assigns each reading to a grid cell.
- **Network class:** builds the 12 × 12 junction graph for a scenario, learns cell averages of speed, delay, CO_2 and fuel (cells with fewer than 3 readings use the scenario average), creates the edges and provides the heuristic and the blocked-road filter.
- **solve:** implements A*, Dijkstra, BFS and Greedy. A* and Dijkstra use a priority queue (heapq), a best-cost map and a parent map; BFS uses a queue; the function returns the path, junctions expanded, travel time, distance, CO_2, fuel and runtime.
- **benchmark:** runs all four algorithms on 300 random trips per scenario (random seed 42, trips longer than 30 km straight-line) and checks each result against the Dijkstra optimum.

- **fig_* functions:** create the five charts (dataset overview, correlation and GPS map, route comparison, benchmark, accident re-routing) and save them with benchmark_results.csv and summary.txt in a results folder.

Table 1. Program settings

Setting	Value	Meaning
GRID	12	12 × 12 junctions over the GPS area
MIN_OBS	3	minimum readings for a cell to use its own averages
TRIALS	300	random trips per scenario in the benchmark
SEED	42	random seed for reproducible trips

5. Dataset and Test-Case Preparation

5.1 Dataset

The experiments use the Sustainable Vehicle Routing dataset (sustainable_vehicle_routing.csv, downloaded from Kaggle). Each record is a GPS reading with a timestamp, latitude, longitude, speed, traffic density (Low, Medium, High), road condition (Dry, Wet, Icy), weather condition (Clear, Rainy, Foggy, Stormy), fuel consumption, CO_2 emission, travel time and an is_optimal_route flag.

Table 2. Dataset summary after cleaning

Item	Value
Records (after removing duplicates and missing values)	3,565 (no duplicates or missing values found)
Negative fuel values corrected	2 (set to 0)
Traffic density: Low / Medium / High	1,047 / 1,839 / 679 readings
GPS area	latitude 35.0–36.0, longitude −120.0 to −119.0
Observed speed range	approximately 13 to 80 km/h

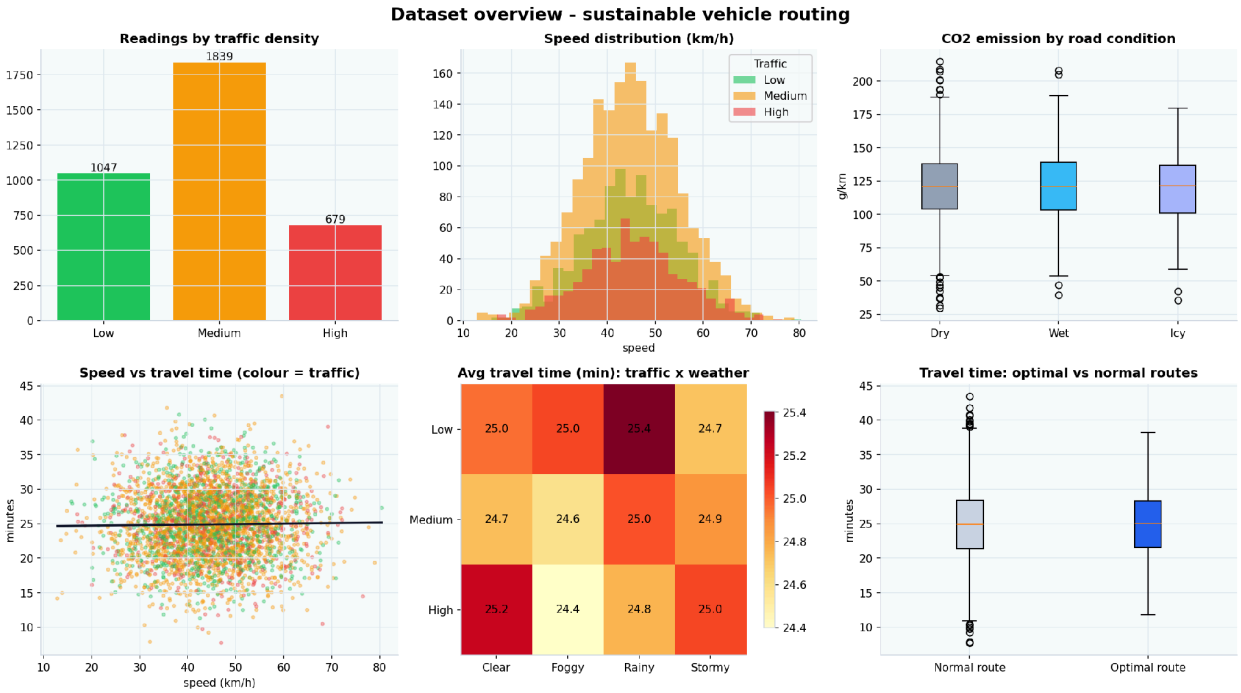


Figure 1. Dataset overview: traffic distribution, speed, CO_2 by road condition, speed vs travel time, travel time by traffic and weather, and optimal vs normal routes.

Observation about the data. The charts and the correlation matrix (Figure 2) show almost no linear relationship between the recorded variables: the correlation between speed and travel time is 0.01 and between the delay penalty and travel time is −0.01, and the mean travel time varies only between 24.4 and 25.4 minutes across all traffic and weather combinations. The only noticeable relationships involve the is_optimal_route flag (−0.40 with the delay penalty and −0.21 with fuel consumption and CO_2). The dataset therefore supplies realistic positions, speeds, fuel and emission values, but the effect of traffic, road and weather on travel time in this project comes from the delay penalties of Table 3 rather than being learned from the data.



Figure 2. Correlation between features (left) and GPS readings coloured by speed over the 12 × 12 junction grid (right).

5.2 Network Construction and Delay Penalties

The GPS area is divided into 12 × 12 equal cells. The centre of each cell is a junction, and each junction is connected to its horizontal, vertical and diagonal neighbours. The average speed, delay penalty, CO_2 and fuel of the readings falling in a cell are attached to that junction. The delay penalty of one reading is the sum of the three components in Table 3.

Table 3. Delay penalty values (added to the factor 1.0)

Condition	Level	Penalty
Traffic density	Low / Medium / High	0.00 / 0.25 / 0.60
Road condition	Dry / Wet / Icy	0.00 / 0.10 / 0.30
Weather	Clear / Rainy / Foggy / Stormy	0.00 / 0.10 / 0.15 / 0.25

5.3 Scenarios

- **All traffic:** all 3,565 readings are used.
- **Low traffic:** only readings with Low traffic density (1,047).
- **Medium traffic:** only readings with Medium traffic density (1,839).
- **High traffic:** only readings with High traffic density (679).

Each scenario produces its own network with the same junctions and roads but different speeds, delays, CO_2 and fuel values.

5.4 Test Cases

- **Benchmark trips:** 300 random start–destination pairs per scenario (seed 42, at least 30 km apart in a straight line), each solved by all four algorithms.
- **Optimality check:** the travel time of every result is compared with the Dijkstra optimum.
- **Demonstration trip:** a 45–90 km trip in the High-traffic scenario, selected automatically as one where the algorithms produce clearly different routes.
- **Accident test:** two connected roads in the middle of the A* demonstration route are blocked and the route is recomputed.
- **Disconnected destination:** if blocked roads cut off the destination, the search returns no route.

6. Performance Evaluation

Running the program prints the benchmark table and saves it to benchmark_results.csv. The reported metrics are:

- **Travel time (minutes):** sum of the edge costs along the route; lower is better.
- **Junctions expanded:** number of junctions removed from the frontier and expanded; lower means less search effort.
- **Runtime (milliseconds):** measured with time.perf_counter(); it is very small on a 144-junction graph and sensitive to machine load.
- **CO_2 (g) and fuel (L):** emission and consumption along the route, from the dataset averages.
- **Optimal trips (%):** share of trips whose travel time equals the Dijkstra optimum.

Table 4. Benchmark results (average over 300 random trips per scenario)

Scenario	Algorithm	Time (min)	Distance (km)	CO_2 (g)	Junctions expanded	Runtime (ms)	Optimal (%)
All traffic	A*	113.3	65.1	7881	35.6	0.271	100.0
All traffic	Dijkstra	113.3	65.1	7881	89.2	0.518	100.0
All traffic	BFS	130.7	73.5	8881	89.7	0.255	15.7
All traffic	Greedy	115.8	65.1	7860	7.6	0.075	26.7
Low traffic	A*	90.1	63.9	7767	28.3	0.212	100.0
Low traffic	Dijkstra	90.1	63.9	7767	86.5	0.471	100.0
Low traffic	BFS	104.4	71.2	8666	85.8	0.229	16.0
Low traffic	Greedy	93.3	63.7	7740	7.4	0.069	26.7
Medium traffic	A*	112.2	63.7	7656	37.2	0.268	100.0
Medium traffic	Dijkstra	112.2	63.7	7656	85.6	0.470	100.0
Medium traffic	BFS	127.5	70.8	8517	85.8	0.230	17.3
Medium traffic	Greedy	114.6	63.7	7651	7.4	0.074	32.7
High traffic	A*	139.8	64.3	7760	47.1	0.313	100.0
High traffic	Dijkstra	139.8	64.3	7760	86.8	0.460	100.0
High traffic	BFS	161.6	71.7	8692	86.5	0.223	18.7
High traffic	Greedy	144.2	64.1	7784	7.5	0.067	32.0

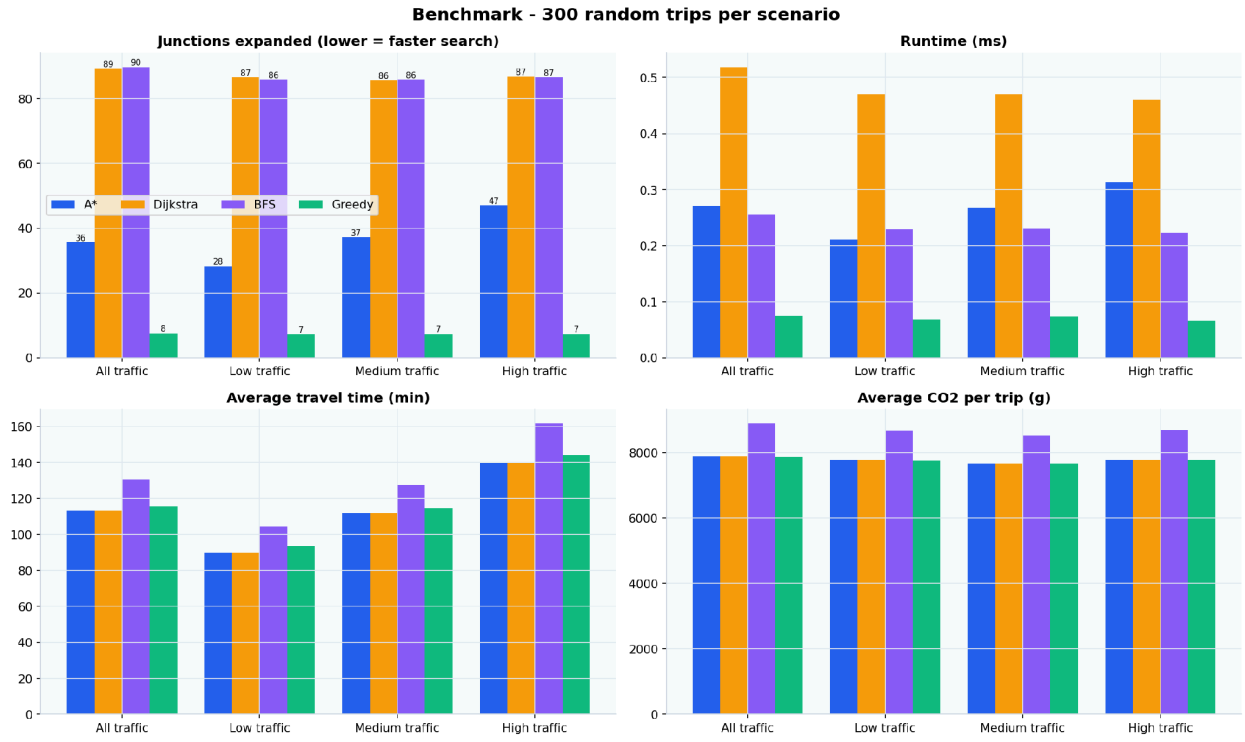


Figure 3. Benchmark of the four algorithms: junctions expanded, runtime, travel time and CO_2 for 300 random trips per scenario.

7. Comparison with Alternative Approaches

A* is compared with Dijkstra (the optimal uninformed baseline), BFS (traffic-unaware) and greedy search (heuristic only). All four run on exactly the same network and trips.

Table 5. Qualitative comparison

Algorithm	Uses travel-time cost	Uses heuristic	Optimal	Behaviour
A*	Yes	Yes	Yes (admissible heuristic)	Optimal route with fewer expansions

Algorithm	Uses travel-time cost	Uses heuristic	Optimal	Behaviour
Dijkstra	Yes	No	Yes	Optimal route, expands in all directions
BFS	No	No	No (fewest junctions only)	Ignores traffic; slower, higher CO_2
Greedy	No (only h)	Yes	No	Very few expansions; often suboptimal

Table 6. Scenario-wise comparison against A*

Scenario	A* junctions	Dijkstra junctions	A* saving	BFS extra time	BFS extra CO_2	Greedy extra time
All traffic	35.6	89.2	60.0%	15.4%	12.7%	2.2%
Low traffic	28.3	86.5	67.2%	15.8%	11.6%	3.6%
Medium traffic	37.2	85.6	56.6%	13.7%	11.2%	2.1%
High traffic	47.1	86.8	45.8%	15.6%	12.0%	3.1%

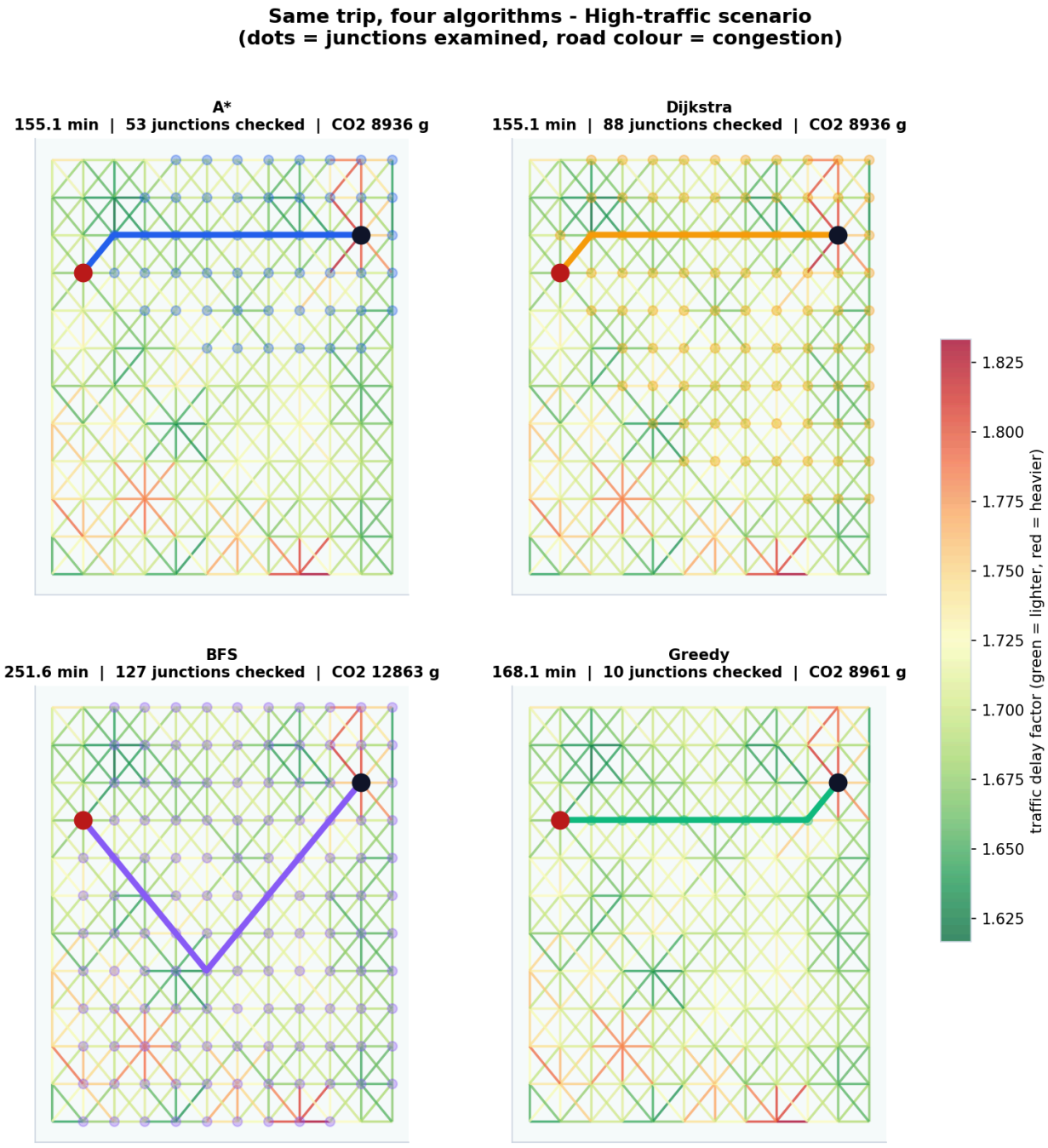


Figure 4. The same High-traffic trip solved by A*, Dijkstra, BFS and Greedy. Dots show the junctions examined; road colour shows congestion.

In the demonstration trip, A* and Dijkstra both found the optimal route of 155.1 minutes, but A* examined 53 junctions and Dijkstra 88. BFS examined 127 junctions and returned a route of 251.6 minutes with 12,863 g of CO_2 (against 8,936 g for A*), because it chose the route with the fewest junctions instead of the fastest. Greedy search examined only 10 junctions but produced a 168.1-minute route.

8. Results, Discussion, and Conclusion

8.1 Results

- **Correctness:** A* matched the Dijkstra travel time in 100% of the 1,200 benchmark trips, confirming that the heuristic is admissible.
- **Search effort:** A* expanded 45.8%–67.2% fewer junctions than Dijkstra in the four scenarios (57% on average). The saving was largest in the Low-traffic network (67.2%) and smallest in the High-traffic network (45.8%); the heuristic ignores traffic delays, so it guides the search less when delays are heavier.
- **Runtime:** A* ran 32.0%–55.0% faster than Dijkstra. BFS and Greedy had lower or similar runtimes because they avoid some priority-queue work, but BFS gives worse routes. All runtimes are below 0.6 ms, so they are only indicative.
- **Traffic sensitivity:** the optimal travel time grew from 90.1 minutes in the Low-traffic network to 139.8 minutes in the High-traffic network, an increase of 55%, showing that the route cost responds to traffic conditions.
- **Traffic-unaware planning:** BFS trips took 13.7% to 15.8% longer and emitted 11.2% to 12.7% more CO_2 than A*, and BFS found the optimal route in only 16–19% of trips.
- **Greedy search:** found the optimal route in 27–33% of trips; its routes were 2.1% to 3.6% slower than optimal.

8.2 Accident Re-routing

Two connected roads on the middle of the A* route of the demonstration trip were blocked to simulate an accident. A* found a new route around the blocked roads, and the travel time increased from 155.1 to 159.7 minutes (+4.5 minutes), showing that the planner adapts to road closures at a small cost in time.

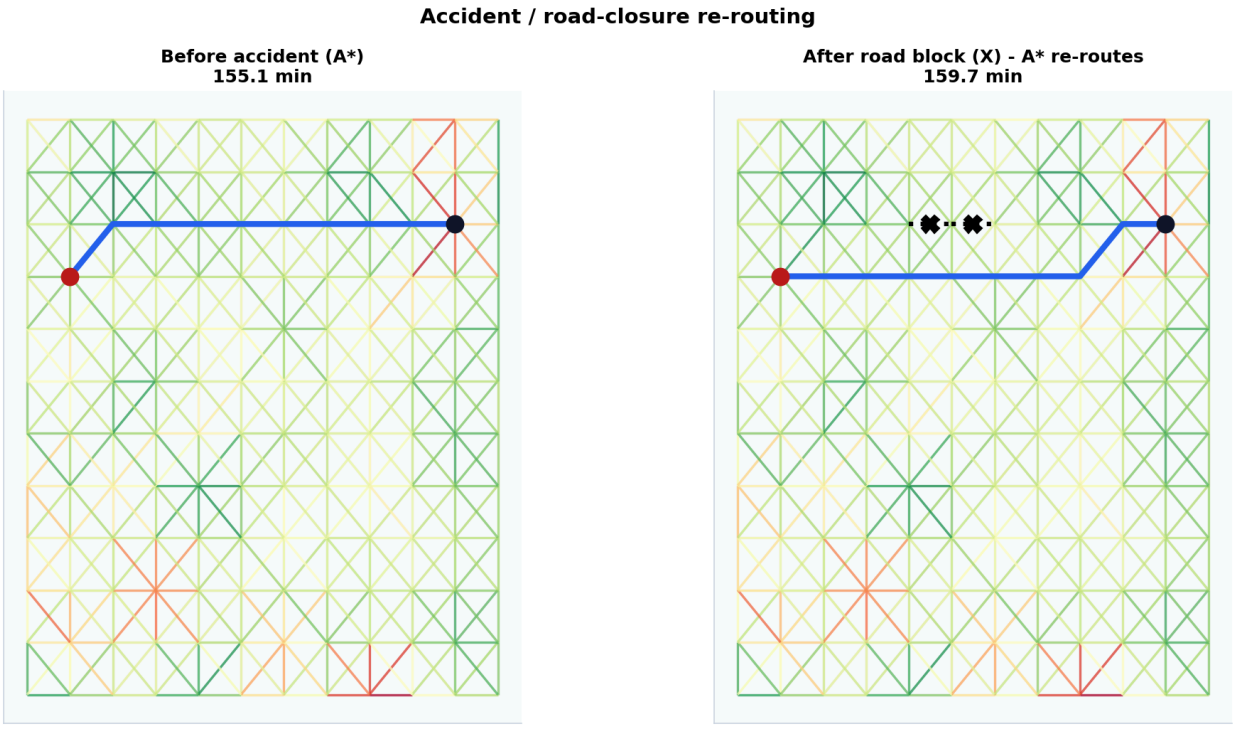


Figure 5. Route before the accident (left) and after the two roads marked X were blocked (right).

8.3 Discussion and Limitations

- The traffic, road and weather effects come from assumed penalty values (Table 3), because the dataset itself shows almost no relationship between these variables and speed or travel time (Section 5.1). Different penalties would change the absolute times but not the correctness of A*.
- The network is a grid abstraction of the GPS area, not a real road map; roads connect neighbouring cells and one-way streets, signals and road classes are not modelled.
- Estimated trip times (90–140 minutes for about 65 km) are model values built from cell-average speeds and delay factors; they should not be compared with the travel_time column of the dataset, which has a mean of about 25 minutes.
- CO_2 is treated as grams per km and fuel as litres per km, which is an interpretation of the dataset columns.
- Runtimes on a 144-junction graph are in fractions of a millisecond, so the number of expanded junctions is a more reliable indicator of search effort than runtime.

8.4 Future Work

Possible improvements are using a real road network with time-of-day speed data, learning the delay penalties from a dataset in which traffic actually affects speed, adding one-way roads and turn costs, optimizing a combined time–CO_2 objective, testing larger graphs with repeated runtime trials, and re-planning dynamically while the vehicle is moving.

8.5 Conclusion

This project shows that A* with an admissible heuristic solves traffic-aware route planning on a network built from vehicle GPS data. It always returned the same optimal route as Dijkstra while examining on average 57% fewer junctions, whereas BFS ignored traffic and produced slower, higher-emission trips and greedy search was fast but rarely optimal. The route cost reacted to traffic conditions and A* re-routed correctly after a road block. The results also show the importance of data quality: the dataset used here does not contain a measurable traffic–speed relationship, so a dataset with real congestion effects would make the evaluation more convincing.

9. Demonstration of the Working Prototype

Live project link: the working prototype is also available online at https://traffic-aware-astar.vercel.app/.

Place traffic_route_optimization.py and sustainable_vehicle_routing.csv in the same folder, install the dependencies and run the program:

```
pip install pandas numpy matplotlib
python traffic_route_optimization.py
```

The program prints the cleaning report, the network size and the benchmark table, then opens the five charts one after another and prints the key findings. Everything is also saved in a results folder: 1_dataset_overview.png, 2_correlation_and_gps_map.png, 3_route_comparison.png, 4_benchmark.png, 5_accident_rerouting.png, benchmark_results.csv and summary.txt. The option --no-show saves the files without opening the chart windows. In the Jupyter/Colab version the same steps are run cell by cell with the dataset in the working folder. The key findings printed by the program were:

- A* is optimal in 100% of trips (same travel time as Dijkstra) but checks 57% fewer junctions.
- BFS ignores traffic: its trips take 15.1% longer and emit 11.9% more CO_2 than A*.

- Greedy search is fast but optimal in only 30% of trips.
- Demo trip (High traffic): 155.1 min; after a road block A* re-routes to 159.7 min (+4.5 min).

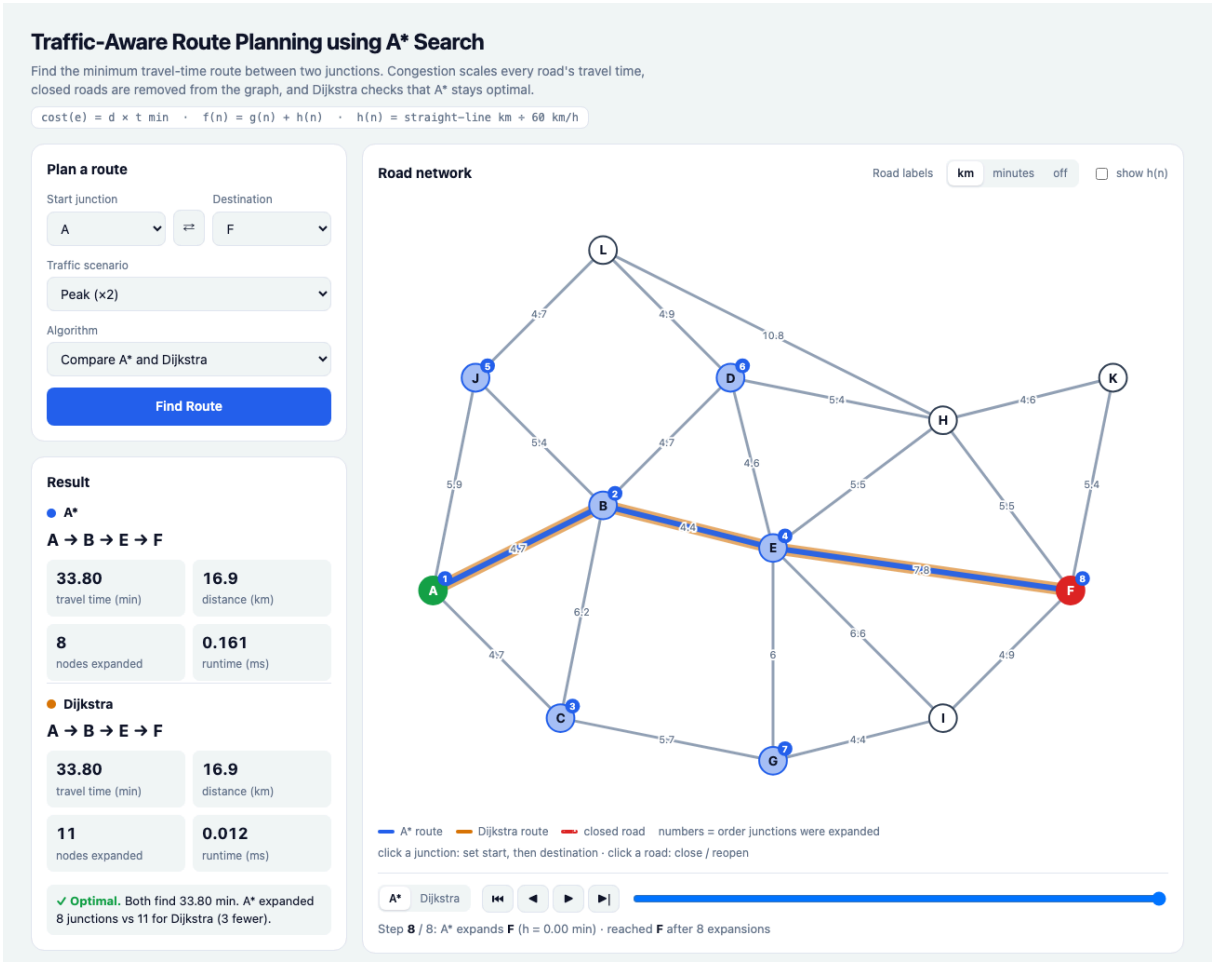


Figure 1. Peak scenario, A → F: both algorithms return A-B-E-F (33.80 min). A expanded 8 junctions and Dijkstra 11.*

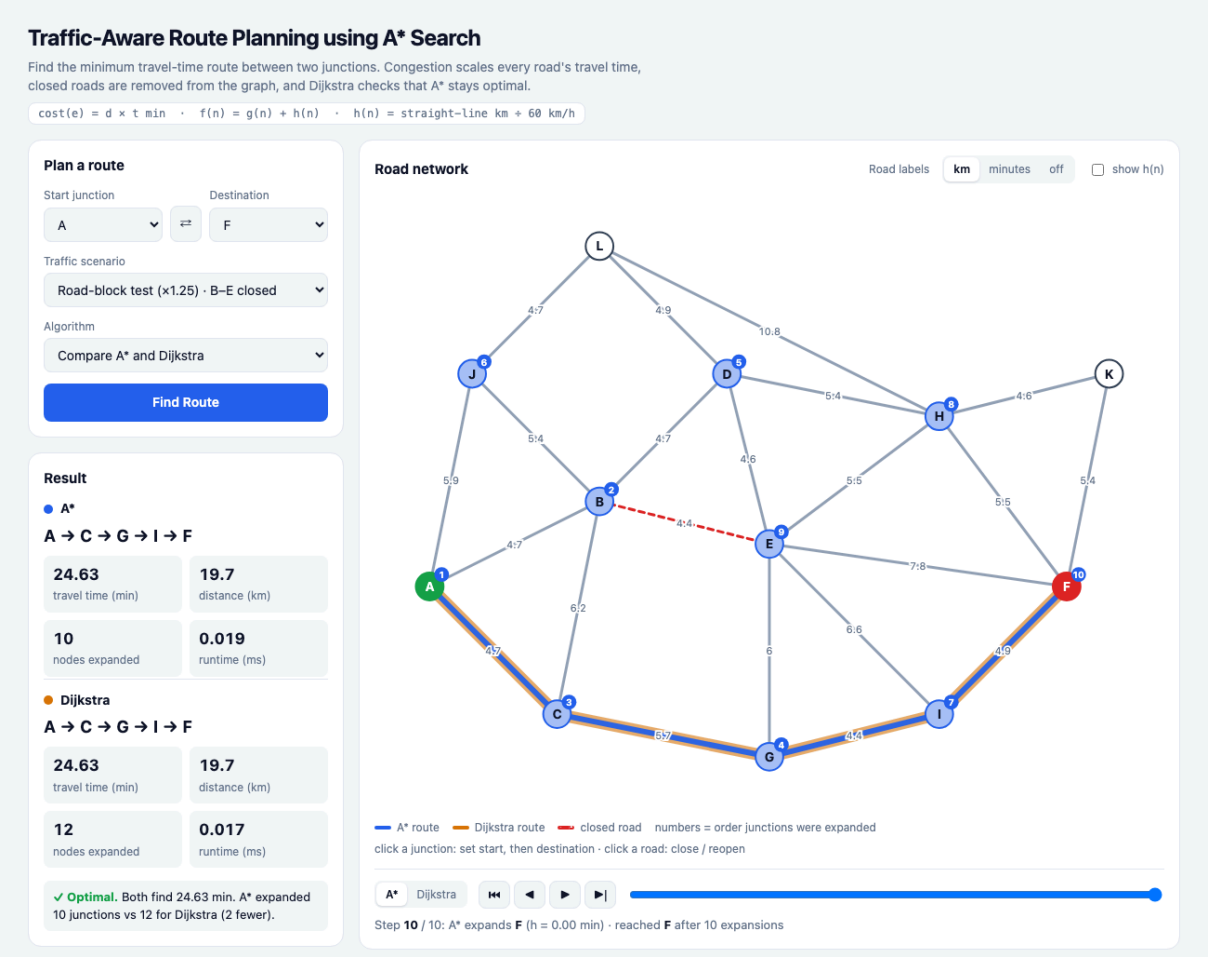


Figure 2. Road-block test: with B–E closed (dashed red), both algorithms re-route via A-C-G-I-F (24.62 min). A expanded 10 junctions and Dijkstra 12.*

References

- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271.
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Sustainable Vehicle Routing dataset (sustainable_vehicle_routing.csv). Kaggle.
- McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
- Python Software Foundation. Python documentation: heapq — Heap queue algorithm.

Appendix A: Project File Structure

```
traffic_route_project/
```

```
├── traffic_route_optimization.py   (or Traffic_route_optimization.ipynb)
├── sustainable_vehicle_routing.csv
└── results/
    ├── 1_dataset_overview.png
    ├── 2_correlation_and_gps_map.png
    ├── 3_route_comparison.png
    ├── 4_benchmark.png
    ├── 5_accident_rerouting.png
    ├── benchmark_results.csv
    └── summary.txt
```

Project URL: https://traffic-aware-astar.aathan.in/

Appendix B: Key Code Extracts

Edge cost, heuristic and A* priority (from traffic_route_optimization.py):

```
km = math.dist(pos[u], pos[v])
spd, f = mean_speed(u, v), 1 + mean_delay(u, v)
minutes = km / spd * 60 * f                      # edge cost

def h(n, goal):                                   # admissible heuristic
    return math.dist(pos[n], pos[goal]) / vmax * 60

ng = g[u] + e["min"]
heapq.heappush(heap, (ng + h(v, dst), v))        # A*: f = g + h
heapq.heappush(heap, (ng, v))                    # Dijkstra: f = g
```